BASIC NETWORK TRAFFIC ANALYSIS (WIRESHARK + NMAP

VIVEK AHIR

Contents

Introduction

Understanding how network communication works at the packet level is an essential skill in cybersecurity, especially for tasks related to monitoring, analysis, and threat detection. Network traffic analysis allows security professionals to observe how data moves across a network, identify normal behavior, and detect patterns that may indicate potential security risks.

In this lab, Wireshark was used to capture and analyze live network traffic in a Kali Linux environment. Different types of traffic were intentionally generated using tools such as ping, curl, and Nmap in order to simulate real-world network activity. These actions produced ICMP, HTTP, DNS, and TCP packets, which were then examined using Wireshark filters. The main objective of this lab was to gain practical experience in packet analysis, understand how different protocols operate, and identify both normal and potentially suspicious behavior within network traffic. By analyzing the captured data, it was possible to observe key networking processes such as connectivity testing, domain name resolution, TCP handshakes, and basic reconnaissance techniques like port scanning.

I. Wireshark Interface

Before starting the packet capture, Wireshark was opened in the Kali Linux environment to select the appropriate network interface. At this stage, no traffic is being captured yet, and Wireshark is simply waiting for the user to choose an interface. Among the available options, the eth0 interface was selected because it represents the main network connection used by the system to communicate with external networks. This is important because choosing the correct interface ensures that all relevant traffic, such as internet activity and generated packets, can be captured properly.

Other interfaces like loopback (lo) and monitoring interfaces are also shown, but they are typically used for internal or specialized traffic. For this lab, eth0 is the most appropriate choice since the goal is to analyze real network communication. This step is essential because if the wrong interface is selected, the captured data may be incomplete or irrelevant to the analysis.

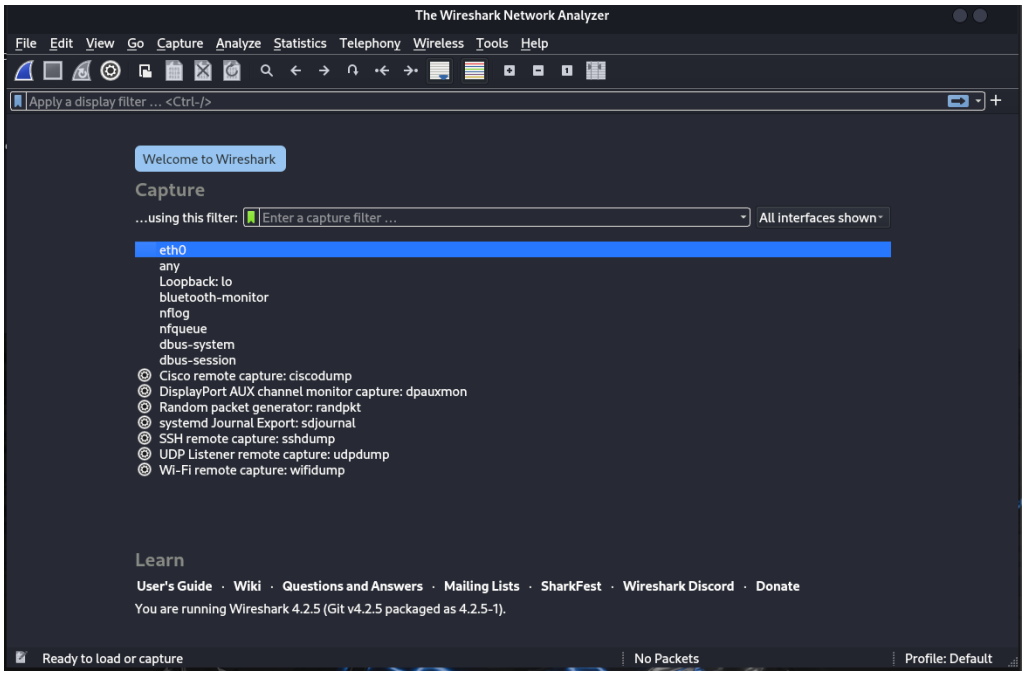

Figure 1: Selection the interface eth0

After selecting the eth0 interface, the packet capture process was started in Wireshark. At this point, Wireshark begins listening to the network and is ready to capture any incoming or outgoing traffic. In the image, the capture has already started, but no packets are visible yet. This is because no network activity has been generated at this moment. The packet list section remains empty until some form of communication occurs on the network. This step confirms that Wireshark is actively monitoring the selected interface and is prepared to record traffic once it is generated. It is an important step because it ensures that all subsequent network activity, such as ping requests or web traffic, will be captured for analysis.

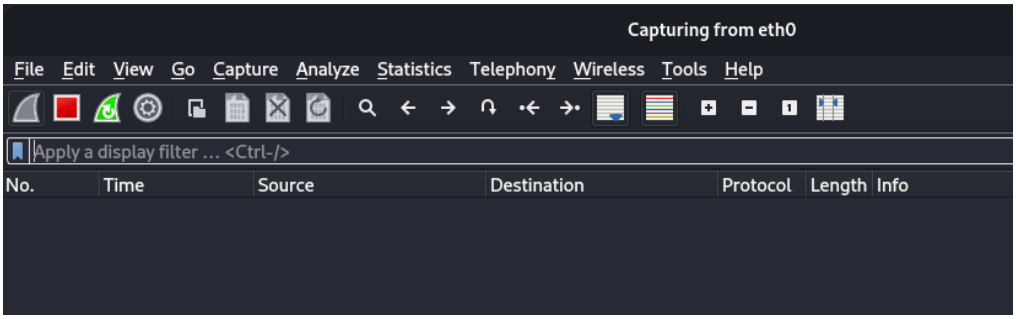

Figure 2: Start capturing traffic

II. Generating Traffic

After starting the packet capture, network traffic was generated using the ping google.com command in the terminal. This was done to create ICMP traffic that could be captured and analyzed in Wireshark. The terminal shows multiple responses from the Google server, indicating that the system is successfully sending and receiving packets. Each line represents an ICMP Echo Reply, along with details such as sequence number and response time.

At the same time, Wireshark is capturing this activity in real-time. The captured packets appear as ICMP protocol entries, showing both Echo Requests sent from the local machine and Echo Replies received from the destination. This step is important because it demonstrates basic network connectivity and provides a clear example of how ICMP traffic looks in packet analysis. It also helps in understanding how request and response communication works at the network level.

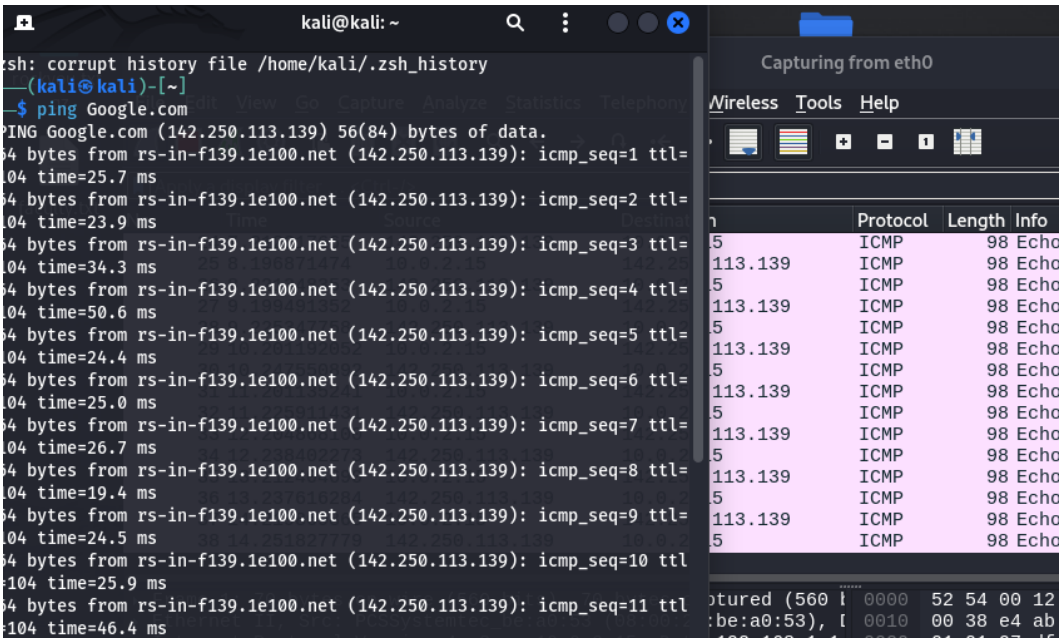

Figure 3: Ping to Google.com

After analyzing ICMP traffic, HTTP traffic was generated using the command curl http://example.com in the terminal. This command sends an HTTP request to the server and retrieves the web page content. In the terminal, the HTML code of the webpage is displayed, confirming that the request was successful and a response was received from the server. This indicates proper communication between the client and the web server over HTTP.

At the same time, Wireshark captures this activity and displays it as HTTP packets. These packets include the HTTP GET request sent by the client and the corresponding response from the server. The traffic is shown over port 80, which is the standard port used for HTTP communication. This step is important because it demonstrates how web traffic appears at the packet level and helps in understanding how HTTP requests and responses are structured and transmitted over the network.

Figure 4: Generating HTTP traffic

III. Captured Network Traffic Overview

After generating different types of traffic, Wireshark began displaying all captured packets in a single view. This screenshot shows a combination of ICMP, DNS, TCP, and HTTP traffic captured during the lab. At the top, ICMP packets from the ping command can be observed, including both Echo Requests and Echo Replies between the local machine and the Google server. This confirms that the earlier connectivity test was successfully captured.

Following that, DNS traffic is visible, where the system sends queries to resolve domain names such as example.com into IP addresses. This step is necessary before establishing any web communication. Below the DNS traffic, TCP packets are shown, including SYN, SYN-ACK, and ACK flags, which represent the process of establishing a connection between the client and the server. This is part of the TCP three-way handshake. Finally, HTTP packets are visible, including GET requests and server responses such as “200 OK,” indicating successful communication with the web server.

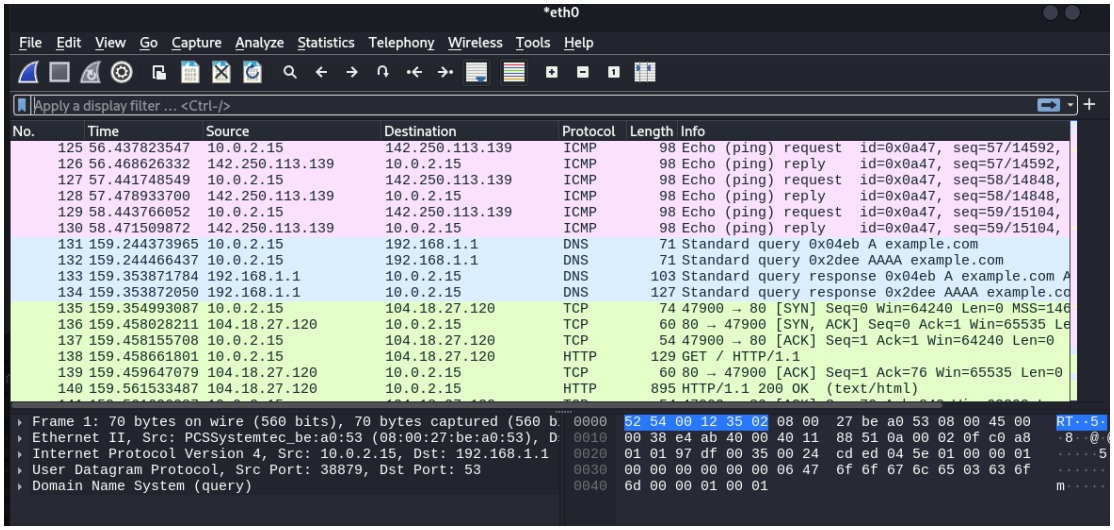

Figure 5: Captured Network Traffic

IV. ICMP Traffic Analysis Using Filter

To focus specifically on ICMP traffic, the filter icmp was applied in Wireshark. This allowed only ICMP packets to be displayed, making it easier to analyze the ping activity generated earlier. From the filtered results, both Echo Request and Echo Reply packets can be clearly observed. The local machine (10.0.2.15) sends Echo Requests to the destination (142.250.113.139), and the server responds with Echo Replies. This confirms successful two-way communication between the client and the server. Each packet includes additional details such as sequence number and time intervals, which indicate how consistently the packets are being sent and received. The regular pattern of requests and replies shows stable network connectivity. This step is important because it isolates ICMP traffic and helps in understanding how basic network diagnostics work. It also demonstrates how filters in Wireshark can be used to simplify packet analysis by focusing on a specific protocol.

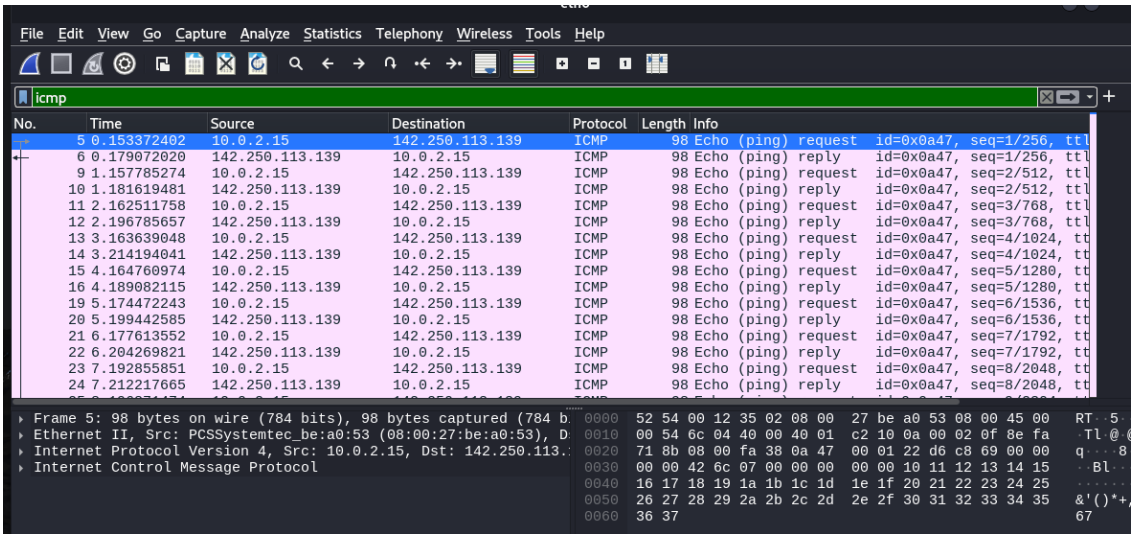

Figure 6: ICMP filter

V. HTTP Traffic Analysis Using Filter

To analyze web traffic more clearly, the filter http was applied in Wireshark. This allowed only HTTP-related packets to be displayed, making it easier to focus on the communication generated by the curl command. From the filtered results, an HTTP GET request can be observed from the local machine (10.0.2.15) to the destination server (104.18.27.120). This request represents the client asking the server for the webpage content.

In response, the server sends back HTTP responses, including “200 OK”, which indicates that the request was successful and the content was delivered properly. Additionally, a “301 Moved Permanently” response can be seen, which shows that the server redirected the request to another location. This is common behavior for many websites. The lower panel of Wireshark also shows the detailed structure of the HTTP packet, including protocol headers and raw data. This helps in understanding how web communication is actually transmitted over the network. This step is important because it demonstrates how

HTTP requests and responses appear at the packet level and highlights how filters can be used to isolate and analyze specific types of traffic.

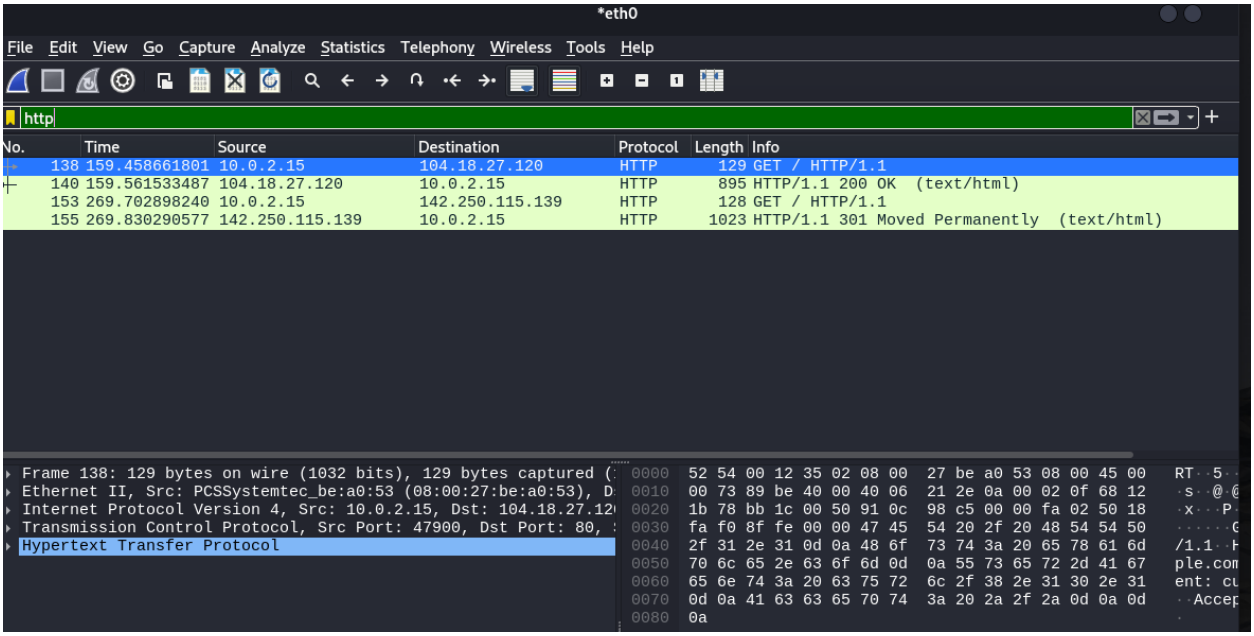

Figure 7: HTTP filter

VI. TCP Traffic Analysis Using Filter

To analyze connection-level communication, the filter tcp was applied in Wireshark. This displays all TCP packets, which are responsible for establishing and maintaining reliable communication between systems. From the filtered results, the TCP three-way handshake can be observed. The process begins with a SYN packet sent from the local machine (10.0.2.15) to the destination server (104.18.27.120), indicating a request to establish a connection. The server then responds with a SYN-ACK packet, acknowledging the request and signaling its readiness to communicate. Finally, the client sends an ACK packet, completing the handshake and establishing the connection.

After the connection is established, HTTP traffic is exchanged over this TCP session, which can also be seen in the captured packets. Additional TCP packets such as ACK and FIN can be observed, indicating ongoing communication and proper session termination. This step is important because it demonstrates how reliable connections are established in networks and helps in understanding how data transfer begins before higher-level protocols like HTTP are used.

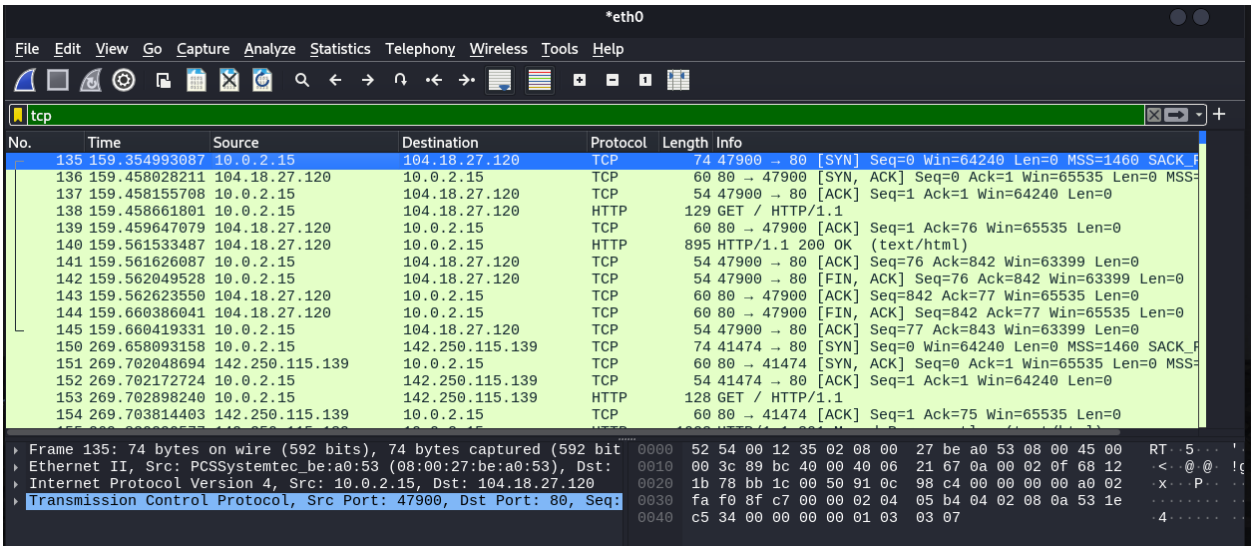

Figure 8: TCP filter

VII. Detection of Suspicious Traffic (SYN Scan Behavior)

To identify potentially suspicious network activity, a more specific filter was applied in Wireshark:

tcp.flags.syn == 1 and tcp.flags.ack == 0.

This filter displays only TCP packets where the SYN flag is set but the ACK flag is not, which typically represents the initial step of a connection attempt. From the filtered results, multiple SYN packets can be observed being sent from the local machine (10.0.2.15) to different external IP addresses. These packets are attempting to initiate connections but do not show the complete handshake process. This behavior is commonly associated with port scanning, particularly SYN scans, which are often used during the reconnaissance phase of a cyber attack. Instead of completing full connections, the scanner sends multiple SYN packets to identify which ports are open or responsive.

In this case, the traffic was intentionally generated using Nmap; however, in a real-world scenario, similar patterns could indicate malicious activity. Being able to detect this type of behavior is important for security monitoring and intrusion detection. This step highlights how Wireshark filters can be used not only to analyze normal traffic but also to identify patterns that may represent potential threats.

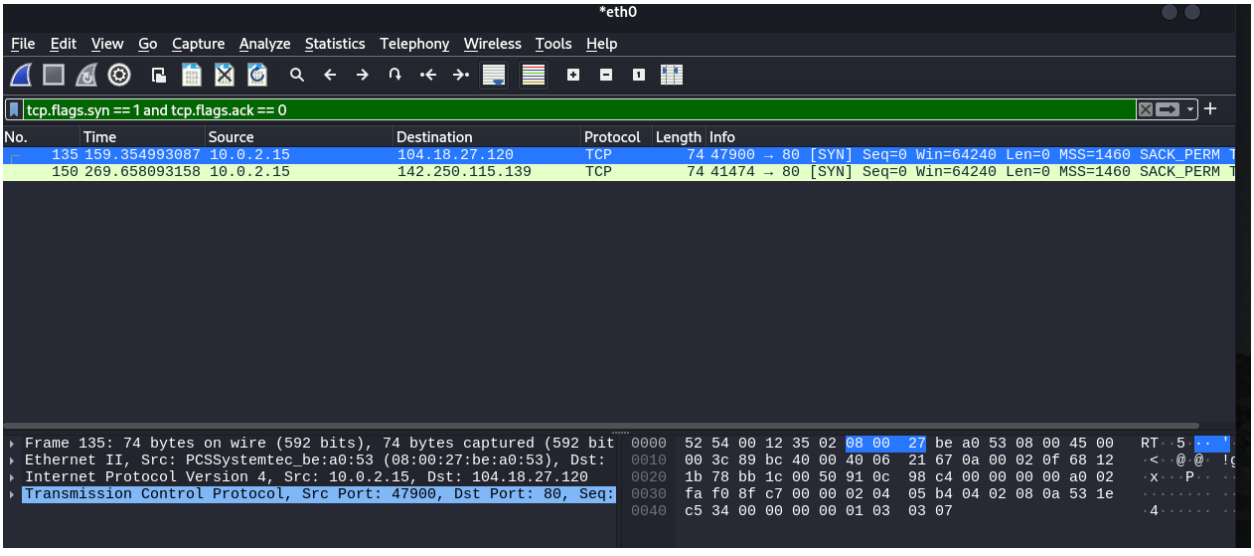

Figure 9: Filtering suspicious traffic

Conclusion

This lab provided hands-on experience in capturing and analyzing network traffic using Wireshark in a Kali Linux environment. By generating different types of traffic such as ICMP, HTTP, and TCP, it was possible to observe how common network protocols operate and interact with each other in real time.

Through the analysis, normal network behavior was clearly identified, including ICMP echo requests and replies for connectivity testing, DNS queries for domain resolution, and the TCP three-way handshake used to establish reliable connections. Additionally, HTTP communication was examined to understand how web requests and responses are transmitted over the network. An important part of this lab was the identification of suspicious patterns, specifically SYN packets without corresponding ACK responses. This behavior, generated using Nmap, demonstrated how port scanning activity appears at the packet level. Recognizing this type of pattern is critical in cybersecurity, as it is commonly associated with reconnaissance attempts.